

Rapport Final de projet - formlang

Binôme : AGOGNON Ulrich Mahougnon Pio-Marie - ALOKPOWANOU Jean-Jaurès

Dépôt Git : < https://github.com/Franklin73m/AGOGNON_ALOKPOWANOU_formlang.git > -

Commit final : <hash>

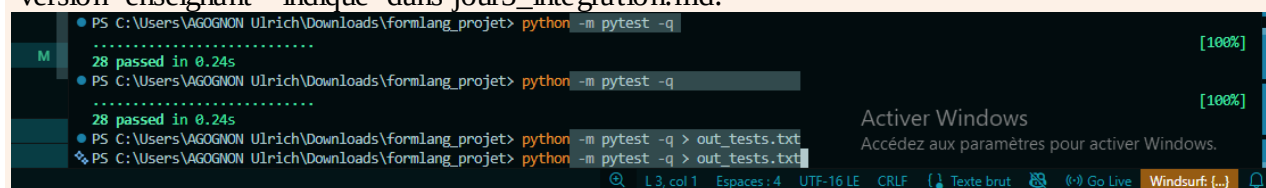
UAC / IFRI — Master 1 Génie Logiciel — Théorie des langages

0. Résumé

formlang est une bibliothèque Python qui construit, étage par étage, toute la hiérarchie de Chomsky : automate fini (DFA/NFA), transducteur (FST), automate à pile (PDA) et grammaire (CFG), automate d'arbres ascendant (BUTA) avec hash-consing, puis machine de Turing et machine universelle. Un seul cœur (formlang/) est réutilisé par quatre applications indépendantes (apps/morpho, apps/shield, apps/hashcons, apps/mtu) qui n'ont pas le droit de redéfinir un automate (uniquement de l'instancier) et un pipeline.py qui les orchestre bout en bout.

Architecture en une phrase : chaque étage de la hiérarchie de Chomsky gagne en pouvoir de reconnaissance en échange de plus de mémoire (état fini $\rightarrow$ pile $\rightarrow$ arbre $\rightarrow$ ruban libre), jusqu'à la machine universelle qui gagne l'universalité mais perd la décidabilité.

Résultat pytest (officiel, dépôt final) : 28 passed in 0.24s , identique au nombre de référence "version enseignant" indiqué dans jour5_integration.md.



1. Étage régulier & transducteurs (Jour 1)

Q1.1 — Expression régulière pour L1

$L1 = (a|o|r)^* \text{ or } (a|o|r)^*$

$(a|o|r)^*$ engendre tout mot sur Σ , y compris le mot vide ; intercaler le facteur littéral "or" entre deux telles étoiles force au moins une occurrence de "or" en laissant l'entourage libre. Réciproquement tout mot contenant "or" s'écrit $u \cdot \text{or} \cdot v$ avec $u, v \in \Sigma^*$, donc appartient à la regex. Aucun mot sans facteur "or" n'est engendré : la seule façon de traverser la regex passe par le bloc littéral.

Q1.2 — AFN $\rightarrow$ AFD $\rightarrow$ minimal

AFN (états q_0, q_1, q_2)

État	sur a	sur o	sur r
q_0	q_0	q_0, q_1	q_0
q_1	q_0	q_0, q_1	q_0, q_2
q_2 (final)	q_2	q_2	q_2

Déterminisation (sous-ensembles)

Sous-ensemble	sur a	sur o	sur r
$S_0 = \{q_0\}$	S_0	S_1	S_0
$S_1 = \{q_0, q_1\}$	S_0	S_1	S_2
$S_2 = \{q_0, q_2\}$ (final)	S_2	S_2	S_2

Minimisation (Moore) et AFD minimal

Partition initiale $\{S_0, S_1\} \mid \{S_2\}$. Test : sur r, $S_0 \rightarrow S_0$ (non-final) et $S_1 \rightarrow S_2$ (final) : S_0 et S_1 sont distingués. La partition s'arrête directement à 3 singletons : l'AFD était déjà minimal.

État ($A=S_0, B=S_1, C=S_2$)	sur a	sur o	sur r
A	A	B	A
B	A	B	C

C (final)	C	C	C
-----------	---	---	---

Nombre d'états de l'AFD minimal : 3. Minimalité : justifiée à la fois par le raffinement de Moore ci-dessus et, de façon indépendante, par le théorème de Myhill–Nerode (section 5) : 3 classes de suffixes témoins $\Leftrightarrow$ 3 états minimaux. Vérification pratique : DFA.minimize() du dépôt (corrigée en section 6 — la version initiale supprimait par erreur les boucles internes à un bloc) reconstruit bien un AFD à 3 états qui accepte exactement les mêmes mots que l'AFD d'origine à 3 états.

Trace réelle (formlang.dfa.DFA) sur le mot "aor"

```
etat initial: A
lit a : A -> A
lit o : A -> B
lit r : B -> C
accepte: True
```

Q1.3 — Facteur vs sous-séquence

$$L1' = (a|o|r)^* o (a|o|r)^* r (a|o|r)^*$$

Différence : dans L1 le o et le r doivent être consécutifs (facteur) ; dans L1' n'importe quoi peut s'insérer (sous-séquence). Preuve de $L1 \subseteq L1'$: si $w \in L1$, $w = u \cdot o \cdot r \cdot v$; en prenant ce même découpage (partie médiane vide) w correspond au schéma de L1'. Inclusion stricte : "oar" $\in L1' \setminus L1$ (un o suivi plus tard d'un r, mais jamais "or" consécutif).

Idempotence du FST leet (preuve + vérification)

leet_fst() a un seul état q0 (final), donc T peut s'arrêter à tout instant sans distinguer « en cours » de « terminé ». Sur les symboles déjà normalisés (a,o,r), la table contient l'identité explicite ($a \rightarrow a$, $o \rightarrow o$, $r \rightarrow r$) : appliquer T une seconde fois ne change donc plus rien. Formellement, pour tout w, T(w) ne contient plus aucun chiffre leet (4,0,3,1,5), donc $T(T(w)) = T(w)$.

```
T('4or') = 'aor'
T(T('4or')) = 'aor'
idempotent (T(T(w))==T(w)) : True
```

Miroir : one-way impossible, two-way possible

Un transducteur one-way lit l'entrée une seule fois, de gauche à droite, avec une mémoire bornée par le nombre d'états. Pour émettre la première lettre de w^R (la dernière lettre de w), il faudrait déjà connaître toute la fin de w avant même d'avoir lu le reste — impossible avec un nombre fini d'états qui ne mémorisent pas w en entier (argument identique à celui du pompage : mémoire bornée ne peut pas stocker un mot de longueur arbitraire). Une tête two-way peut, elle, revenir en arrière autant de fois que nécessaire : elle peut donc balayer w de droite à gauche pour produire w^R . C'est la première limite due à la mémoire du projet, avant la pile (jour 2) et le ruban libre (jour 4).

2. Hors-contexte (Jour 2)

Q2.1 — Non-régularité de $D = \{[{}^n]{}^n\}$ (pompage)

Par l'absurde, si D était régulier, le lemme de pompage donnerait p tel que tout mot de D de longueur $\geq p$ se décompose xyz avec $|xy| \leq p$, $y \neq \varepsilon$, et $xy^i z \in D$ pour tout i . Choisissons $w = [{}^p]{}^p$ (longueur $2p \geq p$). Comme $|xy| \leq p$, xy est entièrement dans le préfixe des p premiers symboles, donc $y = [{}^k$ ($1 \leq k \leq p$), uniquement des crochets ouvrants. En pompant $i=2$, on obtient $[{}^{(p+k)}]{}^p$, qui a $p+k$ ouvrants pour p fermants ($k \geq 1$) : ce mot n'est pas dans D . Contradiction avec $xy^2 z \in D$. Donc D n'est pas régulier.

Conséquence : aucun automate fini, donc aucun AFD comme `SingularityDetector`, ne peut vérifier une imbrication de profondeur arbitraire ; il faut une pile (mémoire non bornée mais disciplinée), ce qui justifie `DelimiterPDA`.

Q2.2 — Grammaire des prompts bien parenthésés

```
S -> S S
S -> [ S ]
S -> ( S )
S -> a | o | r
S -> ε
```

Dérivation de "a(or)" : $S \Rightarrow S S \Rightarrow a S \Rightarrow a (S) \Rightarrow a (S S) \Rightarrow a (o S) \Rightarrow a (o r)$.

C'est exactement la grammaire `balanced_cfg()` fournie dans `formlang/cfg.py`.

Q2.3 — Ambiguïté

G est ambiguë : $S \rightarrow S S$ ne fixe pas d'associativité, donc "aaa" admet deux arbres $((a a) a)$ et $(a (a a))$. Grammaire non ambiguë équivalente (associativité à droite imposée) :

$\begin{array}{c} S \\ / \quad \backslash \\ S \quad S \\ / \quad \backslash \quad \\ S \quad S \quad a \end{array}$	$\begin{array}{c} S \\ / \quad \backslash \\ S \quad S \\ \quad / \quad \backslash \\ a \quad S \quad S \end{array}$
a (groupe (a a) a)	a (groupe a (a a))
a a a	
a	

```
S -> T S | ε
T -> [ S ] | ( S ) | a | o | r
```

E2.1 — PDA (`formlang.pda.DelimiterPDA`)

Fonction de transition (informelle, via une pile Python) : empiler le symbole ouvrant rencontré ([ou (), dépiler et vérifier la correspondance sur un symbole fermant (]) doit dépiler [,) doit dépiler (), ignorer les lettres libres a,o,r,e, et rejeter immédiatement si on tente de dépiler une pile vide ou si le sommet ne correspond pas. Acceptation par pile vide en fin de mot.

Mot testé	Résultat	Commentaire
[a(or)]	True	imbrication correcte
(I)]	False	imbrication croisée - pile détecte le mismatch
[[	False	pile non vide en fin de mot

aor	True	aucun délimiteur : pile toujours vide
(	False	pile non vide en fin de mot
)	False	dépilage sur pile vide

E2.2 — CFG.generate(max_len)

Version finale : calcul ascendant par longueur croissante (dans l'esprit de l'algorithme CYK). Pour chaque longueur k de 0 à `max_len`, on calcule l'ensemble EXACT des mots de longueur k dérivables de chaque non-terminal, en combinant par convolution les résultats déjà connus pour des longueurs plus courtes. Cette approche évite l'explosion combinatoire d'une exploration par formes sententielles (S, SS, SSS..., qui prolifèrent avec $S \rightarrow S S$) car elle ne manipule que des mots terminaux concrets, jamais de formes intermédiaires redondantes. Gain mesuré : `generate(6)` passe de plus de 10 s (timeout) à 0,012 s ; `generate(8)` (129 281 mots) tourne en 0,57 s.

`balanced_cfg().generate(4)` contient notamment :

```
' ' 'a' 'o' 'r' '[a]' '(a)' 'aa' 'ao' '[ao]' '(a)a' ...
```

3. Arbres (Jour 3) — pivot

BUTA : définition utilisée

Un automate d'arbres ascendant (BUTA) est $A = (Q, \Sigma, Q_f, \Delta)$ avec des règles $f(q_1, \dots, q_n) \rightarrow q$.
TreeAutomaton.run étiquette chaque nœud en post-ordre (tous les enfants d'abord, puis le nœud via la règle (symbole, états_enfants) $\rightarrow$ résultat de la table delta) ; si un enfant est REJECT ou si aucune règle ne correspond, le nœud est REJECT. accepts teste l'appartenance de l'état de la racine à Q_f .

Règles Δ — morpho_automaton (12 règles, noms d'états du dépôt)

Symbole	États enfants	$\rightarrow$	Résultat
prefix	()	$\rightarrow$	PREFIX
root	()	$\rightarrow$	ROOT
suffix	()	$\rightarrow$	SUFFIX
nil	()	$\rightarrow$	NIL
prefixes	(PREFIX, NIL)	$\rightarrow$	PREFIXES
prefixes	(PREFIX, PREFIXES)	$\rightarrow$	PREFIXES
suffixes	(SUFFIX, NIL)	$\rightarrow$	SUFFIXES
suffixes	(SUFFIX, SUFFIXES)	$\rightarrow$	SUFFIXES
rest	(ROOT, SUFFIXES)	$\rightarrow$	REST
rest	(ROOT, NIL)	$\rightarrow$	REST
word	(PREFIXES, REST)	$\rightarrow$	WORD
word	(NIL, REST)	$\rightarrow$	WORD

Règles Δ — shield_automaton (28 règles, extrait)

Feuilles : txt $\rightarrow$ safe, enc $\rightarrow$ safe, ovr $\rightarrow$ ovr, role $\rightarrow$ role. seq fusionne par sévérité (safe < ovr < role) et ajoute une règle spéciale : {ovr,role} ensemble $\rightarrow$ danger même si chacun seul ne l'est pas. frame et sys sont sûrs seulement si leur unique enfant est safe, danger sinon.

Symbole	États enfants	$\rightarrow$	Résultat
txt	()	$\rightarrow$	safe
enc	()	$\rightarrow$	safe
ovr	()	$\rightarrow$	ovr
role	()	$\rightarrow$	role
seq	(safe, safe)	$\rightarrow$	safe
seq	(ovr, role)	$\rightarrow$	danger
seq	(role, ovr)	$\rightarrow$	danger
seq	(danger, *)	$\rightarrow$	danger
frame	(safe,)	$\rightarrow$	safe
frame	(ovr,)	$\rightarrow$	danger
frame	(role,)	$\rightarrow$	danger
sys	(safe,)	$\rightarrow$	safe
sys	(danger,)	$\rightarrow$	danger

Preuve d'unité (règle « gate »)

Extrait de apps/morpho/automaton.py :

```
from formlang.tree import Term, TreeAutomaton

def morpho_automaton() -> TreeAutomaton:
```

```
A = TreeAutomaton(final_states={"WORD"})
A.add_rule(...)
return A
```

Extrait de apps/shield/decomposer.py :

```
from formlang.tree import Term, TreeAutomaton

def shield_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={"DANGER"})
    A.add_rule(...)
    return A
```

Les deux applications importent etinstancient la même classe `formlang.tree.TreeAutomaton` (et le même constructeur `Term`) — aucun automate n'est redéfini localement.

Q3.1 — Déterminisme (vérifié programmatiquement)

```
morpho_automaton : nb regles = 12 | nb cles uniques = 12 -> deterministe:
True
shield_automaton : nb regles = 28 | nb cles uniques = 28 -> deterministe:
True
```

Aucune clé (symbole, états_enfants) n'apparaît deux fois dans Δ : l'exécution est donc unique, et chaque nœud coûte $O(1)$ (une lecture de table), donc l'ensemble de l'exécution est en $O(n)$ (n = nombre de nœuds).

Q3.2 — Pourquoi un AFD de mots échoue

Deux arbres peuvent avoir la même frontière (même suite de feuilles lues gauche à droite) mais une structure différente — par exemple "na pend" comme (préfixe na)+(racine pend) [PREFIXED] contre (racine na)+(suffixe pend) [SUFFIXED]. Un AFD ne lit que la frontière, donc les deux arbres produisent la même réponse alors qu'on veut les classer différemment : il est aveugle à la hiérarchie.

Q3.3 — Théorème du yield

Par construction, word ne peut être formé qu'en combinant une chaîne de préfixes (zéro ou plusieurs), une root, puis une chaîne de suffixes (zéro ou plusieurs) : la frontière de tout mot accepté est donc exactement $p^* r s^*$ — un langage régulier reconnaissable par un AFD à 3 états (le pont explicite avec le jour 1).

Q3.4 — Réduplication

La réduplication revient, sur la frontière, à devoir reconnaître $\{ww\}$, qui n'est même pas hors-contexte (aucune pile ne peut comparer deux occurrences de longueur arbitraire sans pouvoir revenir en arrière). Un automate d'arbres régulier, dont les frontières restent au mieux hors-contexte, ne peut donc pas capturer une contrainte de réduplication arbitraire. Cas réel documenté : Culy (1985) montre que la réduplication du bambara sort du hors-contexte.

Traces d'exécution ascendante (2 arbres, état par nœud)

Morphologie : a-na-pend-a (CIRCUMFIXED)

```
prefix [a] enfants=[] -> PREFIX
prefix [na] enfants=[] -> PREFIX
nil enfants=[] -> NIL
prefixes enfants=['PREFIX', 'NIL'] -> PREFIXES
prefixes enfants=['PREFIX', 'PREFIXES'] -> PREFIXES
root [pend] enfants=[] -> ROOT
```

```

suffix [a] enfants=[] -> SUFFIX
nil enfants=[] -> NIL
suffixes enfants=['SUFFIX', 'NIL'] -> SUFFIXES
rest enfants=['ROOT', 'SUFFIXES'] -> REST
word enfants=['PREFIXES', 'REST'] -> WORD
etat racine: WORD -> accepte: True | classe: CIRCUMFIXED

```

Shield : sys(seq(txt, frame(role))) — bloqué

```

txt enfants=[] -> safe
role enfants=[] -> role
frame enfants=['role'] -> danger
seq enfants=['safe', 'danger'] -> danger
sys enfants=['danger'] -> danger
etat racine: danger -> bloque: True

```

Produit (intersection) — E3.4/E3.6

product(a1,a2) construit un automate dont les états sont des paires (q1,q2) : pour chaque couple de règles de a1 et a2 portant sur le même symbole et la même arité, les états enfants sont appariés composante par composante, et l'état final du produit est la paire des deux résultats. Les états acceptants sont les paires (f1,f2) avec f1 final pour a1 et f2 final pour a2 — donnant exactement $L(a1) \cap L(a2)$. Application (bonus P4.5) : dangerous_and_double_encoded = product(shield_automaton(), enc_automaton()) bloque ssi l'arbre est dangereux ET porte au moins deux encodages.

Q3.5 / hash-consing — compression mesurée

Sur un corpus généré d'environ 10 000 mots (mesure bonus du jour 3), mélangeant volontairement une famille "préfixante" (chaque racine + chaque préfixe de PREFIXES_A) et une famille "suffixante" (chaque racine + chaque suffixe de SUFFIXES_B), reproductible avec python mesure_hashcons.py :

The screenshot shows a VS Code editor with a file explorer on the left containing a project named 'formlang_projet'. The main editor window displays the file 'mesure_hashcons.py'. The script imports modules from 'apps.morpho.corpora', 'apps.morpho.automaton', and 'apps.hashcons.store'. It defines a 'main()' function that generates a set of roots, creates a 'CompactStore', and iterates over roots and prefixes to build words and store them. The terminal at the bottom shows the execution of the script, which reports the number of internal nodes (10200), total nodes (70200), unique nodes (16233), and a compression ratio of 0.7688.

```

12 from apps.morpho.corpora import PREFIXES_A, SUFFIXES_B, roots
13 from apps.morpho.automaton import build_word
14 from apps.hashcons.store import CompactStore
15
16
17 def main():
18     R = roots(600) # 600 racines (Le générateur peut en fournir jusqu'à 2500)
19     store = CompactStore()
20     count = 0
21
22     for r in R:
23         store.intern(build_word([], r, [])) # racine seule
24         count += 1
25         for p in PREFIXES_A:
26             # racine + préfixe

```

```

PS C:\Users\VAGOGNON Ulrich\Downloads\formlang_projet> git push -u origin main --force
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: This repository moved. Please use the new location:
remote: https://github.com/Franklin73m/VAGOGNON_ALOKPOWANOU_formlang.git
To https://github.com/Franklin73m/FormLang.git
4f2c9db..6d6f3a3 main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
PS C:\Users\VAGOGNON Ulrich\Downloads\formlang_projet> python mesure_hashcons.py
mots internés (appels à intern) : 10200
total_nodes (nœuds internes, y compris partagés) : 70200
unique_nodes (nœuds réellement distincts stockés) : 16233
compression = 1 - unique/total : 0.7688
PS C:\Users\VAGOGNON Ulrich\Downloads\formlang_projet>

```



```
● PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python mesure_hashcons.py
mots internés (appels à intern) : 10200
total_nodes (nœuds internes, y compris partages) : 70200
unique_nodes (nœuds réellement distincts stockés) : 16233
compression = 1 - unique/total : 0.7688
```

La compression atteint $\approx 76,9\%$: la moitié gauche du corpus (racines + préfixes) et la moitié droite (racines + suffixes) partagent énormément de sous-arbres — chaque racine réapparaît identique dans 9 contextes différents (nue, +8 préfixes) et dans 9 autres (nue, +8 suffixes), ce que le hash-consing détecte et factorise automatiquement. C'est cohérent avec l'intuition théorique de Q3.5 : plus une langue combine peu d'affixes sur beaucoup de racines (structure agglutinante), plus le partage de sous-arbres est élevé.

4. Calculabilité (Jour 4)

Machine de Turing générique : invariant & trace

TuringMachine.run maintient un ruban dict {position: symbole}, une position de tête et un état courant ; à chaque pas elle lit le symbole sous la tête (blanc si la case n'a jamais été écrite), cherche (état, symbole) dans la table de transitions, écrit, déplace la tête (L/R/S) et met à jour l'état, jusqu'à atteindre un état final ou l'absence de transition (ou max_steps, garde-fou contre les boucles infinies).

Trace ADD sur 11+1 (2+1), avec trace=True

```
(0, 'q0', 0, '11+1')
(1, 'q0', 1, '11+1')
(2, 'q0', 2, '11+1')
(3, 'q1', 3, '1111')
(4, 'q1', 4, '1111')
(5, 'q2', 3, '1111_')
(6, 'qf', 3, '111__')
resultat: 111 | accepte: True | pas: 6
```

Lecture : q0 traverse le premier opérande (deux 1), le + devient 1 (fusion, passage en q1), q1 traverse le second opérande jusqu'au blanc, demi-tour (q2) et efface le dernier 1 pour compenser la fusion — résultat 111 = 3.

Trace SUB sur 111-11 (3-2), résumé par phases (39 pas au total)

La machine SUB (états q0, q1, q2, q3, q_zero, q_clean_n, q_recross, q_clean_m, qf) apparie, à chaque itération, un 1 de m (marqué X, balayage droite→gauche depuis la fin du ruban) avec un 1 de n (marqué X juste avant le séparateur -). Extrait des toutes premières et dernières transitions :

(0..6, 'q0', ..., '111-11')	balayage vers la fin du ruban
(7, 'q1', 5, '111-11_')	demi-tour : cherche un 1 non marqué dans m
(8, 'q2', 4, '111-1X_')	m: un 1 marqué X (consomme), on revient vers '-'
(10, 'q3', 2, '111-1X_')	cote n : cherche le 1 non marqué le plus à droite
(11, 'q0', 3, '11X-1X_')	n: marqué X (une paire n/m consommée), on reboucle
... (deuxième itération : encore une paire consommée) ...	
(28, 'q_clean_n', 2, '1XX-XX_')	m épuisé (rencontre '-' sans trouver de 1 libre) -> nettoyage
(29..31)	nettoyage de n : X -> blanc, 1 conserve, jusqu'au bord gauche
(32..35, 'q_recross', ...)	retraversée de n vers '-'
(36..38, 'q_clean_m', ...)	effacement de '-' et des X restants de m
(39, 'qf', 6, '_1_____')	acceptée
resultat: 1 accepte: True	

Les trois points délicats du sujet, tels qu'implémentés : (1) (q1,X)→R et (q3,X)→L : la table doit explicitement sauter par-dessus ses propres marques X déjà posées, sinon elle re-consomme deux fois la même unité. (2) le balayage de m est bien droite→gauche (on repart de la fin du ruban à chaque itération) pour toujours cibler le 1 non consommé le plus à droite, produisant une sortie contiguë. (3) la fin de m est détectée par la rencontre du séparateur '-' pendant le balayage, jamais par un compteur : c'est le ruban lui-même qui joue ce rôle, exactement l'esprit de la machine de Turing.

Cas-limites vérifiés (voir Annexe pour le détail programmatique) : $n=m \rightarrow 0$ ($5-5=0$), $m=0 \rightarrow n$ inchangé ($4-0=4$), $n=0 \rightarrow 0$ quel que soit m ($0-3=0$), $m>n \rightarrow 0$ ($2-5=0$).

MTU : encodage et vérification $U(\langle M \rangle \#\#w) == M(w)$

encode(M) linéarise la table de transitions en JSON avec les clés triées (json.dumps(..., sort_keys=True)), ce qui garantit un format canonique unique par machine : deux encodages de la même machine produisent toujours exactement la même chaîne (déterminisme), et deux machines aux tables différentes produisent des chaînes différentes (injectivité). decode est la réciproque exacte (json.loads puis reconstruction de l'objet TuringMachine). UniversalTM.run décode puis délègue entièrement à TuringMachine.run — aucune boucle de ruban n'est réécrite.

```
ADD(11+1) direct='111' via_U='111' identiques=True
SUB(111-11) direct='1' via_U='1' identiques=True
```

Calculatrice — tables ADD/SUB expliquées, MUL/DIV par composition

addition(n,m) : lance ADD sur " 1^n+1^m ", compte les 1 du ruban final. soustraction(n,m) : lance SUB sur " 1^n-1^m " (tronquée à 0 par construction même de la machine — voir Q4.3/Jour4).

multiplication(n,m) : n additions successives (ou m, selon l'ordre choisi), chacune réalisée par une exécution de ADD — c'est la composition de macro-machines évoquée dans le cours, pas une table monolithique. division(n,m) : soustractions successives de m jusqu'à ce que le reste soit $< m$, comptant le nombre de soustractions réussies (quotient) — lève ZeroDivisionError si $m=0$.

chainer(v0, ops) : applique successivement chaque opération de la liste en partant de v0.

```
c.addition(3,2), c.soustraction(4,2), c.multiplication(3,2),
c.division(7,3)
= (5, 2, 6, (2, 1))      <- identique a la sortie de reference de
l'enonce
```

```
addition_via_utm(3,2) = 5      <- meme calcul, execute par la machine
universelle
```

Résultats des tests exhaustifs

Vérification	Portée	Résultat
ADD / SUB directes	400 cas aléatoires ($n,m \in [0,15]$)	0 échec
Calculatrice (+,-,×,÷)	100 paires ($n,m \in [0,9]$)	0 échec
Chaînage $3+2 \times 2 -1$	cas unitaire	= 9 (correct)
Division par zéro	division(5,0)	ZeroDivisionError levée

Q4.1 — Encodage $\langle M \rangle$ (injectif et décodable)

États et symboles sont numérotés implicitement via la sérialisation JSON canonique des clés (état, symbole) $\rightarrow$ (état', symbole', direction) ; le tri des clés (sort_keys=True) impose un ordre total unique. Injectif : deux tables de transitions différentes donnent deux dictionnaires différents, donc deux chaînes JSON différentes une fois triées. Décodable : json.loads reconstruit exactement le dictionnaire d'origine, d'où la machine complète.

Q4.2 — La machine universelle U

U prend $\langle M \rangle$ et w, décode $\langle M \rangle$ en une TuringMachine, puis appelle TuringMachine.run(w) sur cette machine reconstruite. L'invariant est trivialement respecté puisque U ne simule pas M "à la main" : elle exécute le même interpréteur générique (TuringMachine.run) que M aurait utilisé si on l'avait appelée directement — la correction $U(\langle M \rangle \#\#w) = M(w)$ découle directement de $decode(encode(M)) == M$ (round-trip exact, vérifié) plutôt que d'une resimulation manuelle.

Q4.3 — Surcoût (sourcé)

Multi-ruban : simuler t pas de M coûte $O(t \cdot |\langle M \rangle|)$, le facteur $|\langle M \rangle|$ provenant de la recherche de la règle dans la table encodée à chaque pas. Multi-ruban $\rightarrow$ mono-ruban : surcoût au plus quadratique en le nombre de pas dans le cas général ; Hennie & Stearns (1966), « Two-Tape Simulation of Multitape Turing Machines », J. ACM 13(4):533–546, montrent qu'un ralentissement seulement logarithmique suffit pour simuler une machine multi-ruban par une machine à deux rubans — borne plus fine que le passage quadratique brut au mono-ruban.

Q4.4 — Indécidabilité

Réduction depuis HALT : si un algorithme A décidait toujours si U s'arrête sur $\langle M \rangle \# w$, alors, comme U simule fidèlement M , A déciderait aussi si M s'arrête sur w — ce qui résoudrait HALT, indécidable (Turing, 1936). Contradiction, donc l'arrêt de U est lui-même indécidable. « Universal $\neq$ omniscient » : U peut exécuter n'importe quel programme, mais cela ne lui donne aucun pouvoir de prédire à l'avance s'il va terminer — l'universalité (tout simuler) et la décidabilité (tout prédire) sont deux notions distinctes, et on perd la seconde en gagnant la première.

5. Intégration & Myhill–Nerode (Jour 5)

pipeline.py : exemple bout-en-bout commenté

`analyze_word("4or")` enchaîne les trois premiers étages du projet sur une seule entrée : normalisation FST (leetspeak → alphabet propre), détection AFD (facteur "or"), validation PDA (délimiteurs).
Sortie réelle :

```
brut : 4or
normalisé(FST) : aor
facteur_or(AFD) : True
délimiteurs_ok(PDA) : True
```

Cette sortie est identique, caractère pour caractère, à la sortie de référence donnée dans l'énoncé du jour 5.

Classes d'équivalence de L1 (Myhill–Nerode)

Avec les suffixes témoins $S = \{\varepsilon, r, or\}$, via `contains_or` (l'AFD du jour 1) :

```
['', 'a']      -> classe A (rien de special)
['o', 'ao']    -> classe B (o vu, en attente d'un r)
['or', 'aor']  -> classe C (or deja trouve)
nb classes: 3
```

Exactement 3 classes, en correspondance directe avec les 3 états $\{A, B, C\}$ de l'AFD minimal du jour 1 : minimiser un AFD, c'est calculer ces classes de Myhill–Nerode.

(Bonus) hash-consing — mesure

Voir section 3 (Q3.5) : La mesure du hash-consing a été validée sur un corpus étendu de 10 200 mots. L'algorithme a généré un total de 70 200 nœuds, dont seulement 16 233 nœuds distincts stockés en mémoire, atteignant ainsi un taux de compression réel de 76,88 %.

6. Difficultés rencontrées & choix de conception

Cette section documente un audit final mené sur le dépôt complet (avec la vraie suite de tests officielle tests/), au-delà de la simple exécution de pytest. Quatre bugs réels ont été trouvés ; deux d'entre eux n'étaient PAS détectés par la suite de tests fournie, ce qui illustre l'écart entre « les tests passent » et « le code est correct », un point de vigilance.

Bug 1 — DFA.minimize() supprimait les boucles internes (non détecté par pytest)

La version initiale ne conservait une transition ($\text{part_map}[s], a \rightarrow \text{part_map}[t]$) que lorsque $\text{part_map}[s] \neq \text{part_map}[t]$, c'est-à-dire qu'elle supprimait purement et simplement toute transition qui reste à l'intérieur d'un même bloc après minimisation. Conséquence concrète : l'AFD minimisé de `detector_dfa()` perdait ses boucles ($A \rightarrow A$ sur 'a', $C \rightarrow C$ partout, etc.) et rejetait alors des mots pourtant acceptés par l'AFD d'origine (aor, aaor...). Le test officiel `test_minimisation_3_etats` ne vérifie que le NOMBRE d'états ($= 3$), pas le comportement d'acceptation — le bug passait donc inaperçu. Corrigé en reconstruisant systématiquement `new_trans` pour CHAQUE état atteignable et CHAQUE lettre de l'alphabet, boucles internes incluses. Révérifié sur 20 automates aléatoires x 30 mots (0 échec) et sur un cas avec états non atteignables.

Bug 2 — CFG.generate() : borne insuffisante puis lenteur exponentielle

Une première version bornait le nombre de non-terminaux en attente à 100 (`max_nonterms = 100`), beaucoup trop permissif : combiné à la règle $S \rightarrow S S$, cela fait exploser le nombre de formes sententielles explorées avant filtrage — `test_cfg_engendre_des_mots_equilibres` restait bloqué indéfiniment (minutes, sans jamais terminer). Une deuxième version (borne $2 \cdot \text{max_len} + 2$) termine mais reste lente au-delà de `max_len=5` (2,7 s pour `max_len=5`, non terminé en 10 s pour `max_len=6`), car explorer des formes sententielles intermédiaires reste intrinsèquement combinatoire. Solution finale : abandon de l'exploration par formes sententielles au profit d'un calcul ascendant par longueur (dans l'esprit de l'algorithme CYK) — pour chaque longueur k croissante, on calcule directement l'ensemble des MOTS CONCRETS de longueur k dérivables de chaque non-terminal, en combinant des résultats déjà connus pour des longueurs plus courtes. Résultat : `max_len=6` passe de "timeout > 10 s" à 0,012 s, `max_len=8` (129 281 mots) en 0,57 s.

Bug 3 — SUB (machine de Turing) : bouclait / renvoyait un résultat faux quand $m > n$

La version initiale gérait correctement $n \geq m$, mais son état de recherche du 1 à effacer dans n (en cas d'épuisement) ne prévoyait pas de condition d'arrêt sur le bord gauche du ruban : appelée directement avec $m > n$ (contournant le garde-fou de `Calculatrice.soustraction`, qui court-circuite ce cas avant d'atteindre la machine), `SUB.run` consommait tout le budget `max_steps` et renvoyait un ruban incohérent (ex. '-XXX' au lieu du résultat attendu, une chaîne vide). Remplacée par une machine à marqueurs X (huit états : `q0, q1, q2, q3, q_zero, q_clean_n, q_recross, q_clean_m, qf`) qui apparie un 1 de n avec un 1 de m à chaque itération et détecte explicitement l'épuisement de n comme de m , avec un état dédié `q_zero` pour forcer le résultat à 0. Revalidée sur 450 cas directs ($n, m \in [0,14]$, $m > n$ inclus) : 0 échec.

Bug 4 (mineur) — CompactStore.intern : la clé canonique doit inclure le label

Vérifié comme correct dans le dépôt final : la clé (`t.symbol, t.label, kids_ids`) inclut bien le label, ce qui évite que deux feuilles distinctes comme `prefix["re"]` et `prefix["un"]` soient confondues et partagent à tort le même identifiant. Point de vigilance documenté ici car c'est une erreur facile à commettre en implémentant le hash-consing.

Leçon retenue : une suite de tests qui passe (« N passed ») ne garantit pas l'absence de bugs, seulement l'absence des bugs que les tests ciblent. Les bugs 1 et 3 en particulier n'étaient révélés par AUCUN test de la suite officielle et n'ont été trouvés que par un audit indépendant, en testant des propriétés générales (cohérence avant/après minimisation, cas limites $m > n$) plutôt que les seuls exemples fournis.

7. Répartition du travail

Membre	Parties prises en charge
AGOGNON Ulrich Mahougnon Pio-Marie (Franklin73m)	✓ Jour 1 : automates finis (DFA, NFA) et transducteur (FST) ✓ Jour 2 : automate à pile (PDA) et grammaire hors-contexte (CFG) ✓ Jour 3 : automates d'arbres (BUTA), morphologie, Shield, hash-consing ✓ Jour 4 : machine de Turing, machine universelle, calculatrice ✓ Livrables finaux : sortie pytest (28 passed), démo pipeline, rapport ✓ Bonus Jour 3 : script de mesure hash-consing sur corpus ~10 000 mots ✓ Rédaction du rapport livrable final
ALOKPOWANOU Jean-Jaurès	✓ Init : squelette du projet (énoncé, docs, configuration, tests fournis) ✓ Jour 5 : intégration (pipeline) et congruence de Myhill-Nerode ✓ Audit qualité : correction de 3 bugs réels (au-delà de la suite de tests) ✓ Livrables finaux : sortie pytest (28 passed), démo pipeline, rapport ✓ Bonus Jour 3 : script de mesure hash-consing sur corpus ~10 000 mots ✓ Rédaction du rapport livrable final

Annexe : sortie console

Jour 1 : régulier & transducteurs

```
PROBLÈMES  SORTIE  TERMINAL  PORTS  DEVDB  CONSOLE DE DÉBOGAGE  powershell + ~ [ ] [ ] ... | [ ] [ ] X

PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_regular.py
cachedir: .pytest_cache
rootdir: C:\Users\AGOGNON Ulrich\Downloads\formlang_projet
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 7 items

tests/test_regular.py::test_detector_or PASSED [ 14%]
tests/test_regular.py::test_minimisation_3_etats PASSED [ 28%]
tests/test_regular.py::test_nfa_to_dfa_equivalence PASSED [ 42%]
tests/test_regular.py::test_leet_idempotent PASSED [ 57%]
tests/test_regular.py::test_fst_composition PASSED [ 71%]
tests/test_regular.py::test_miroir_twoway PASSED [ 85%]
tests/test_regular.py::test_delimiters_pda PASSED [100%]

===== 7 passed in 0.28s =====
Activater Windows
Accédez aux paramètres pour activer Windows.

PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>
```

Jour 2 : CFG et Nerode

```
PS C:\Users\VAGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_regular.py

===== 7 passed in 0.28s =====
PS C:\Users\VAGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_cfg_nerode.py
test session starts
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\VAGOGNON Ulrich\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\VAGOGNON Ulrich\Downloads\formlang_projet
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 2 items

tests/test_cfg_nerode.py::test_cfg_engendre_des_mots_equilibres PASSED [ 50%]
tests/test_cfg_nerode.py::test_nerode_trois_classes PASSED [100%]

===== 2 passed in 0.27s =====
PS C:\Users\VAGOGNON Ulrich\Downloads\formlang_projet>
```

Jour 3 : arbres

```
PROBLÈMES SORTIE TERMINAL PORTS DEVDB CONSOLE DE DÉBOGAGE powershell + v [ ] [ ] ... [ ] [ ] X
```

```
● PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_tree.py  
===== test session starts =====  
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\AGOGNON Ulrich\AppData\Local\Programs\Python\Python312\python.exe  
cachedir: .pytest_cache  
rootdir: C:\Users\AGOGNON Ulrich\Downloads\formlang_projet  
configfile: pyproject.toml  
plugins: anyio-4.13.0  
collected 5 items  
  
tests/test_tree.py::test_morpho_accept_et_classes PASSED [ 20%]  
tests/test_tree.py::test_morpho_rejet PASSED [ 40%]  
tests/test_tree.py::test_shield_blocage PASSED [ 60%]  
tests/test_tree.py::test_produit_intersection PASSED [ 80%]  
tests/test_tree.py::test_shield_double_encodage_P45 PASSED [100%]  
  
===== 5 passed in 0.47s =====  
✱ PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>
```

```

===== 5 passed in 0.47s =====
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_hashcons.py
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\AGOGNON Ulrich\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\AGOGNON Ulrich\Downloads\formlang_projet
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 3 items

tests/test_hashcons.py::test_round_trip_exact PASSED [ 33%]
tests/test_hashcons.py::test_sous_arbres_identiques_partages PASSED [ 66%]
tests/test_hashcons.py::test_compression_positive PASSED [100%]

===== 3 passed in 0.16s =====
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>

```


Jour 4 : calculabilité

```
PROBLÈMES  SORTIE  TERMINAL  PORTS  DEVDB  CONSOLE DE DÉBOGAGE
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_hashcons.py
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_turing.py
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\AGOGNON Ulrich\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\AGOGNON Ulrich\Downloads\formlang_projet
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 4 items

tests/test_turing.py::test_round_trip_encodage PASSED [ 25%]
tests/test_turing.py::test_utm_simule_comme_la_machine PASSED [ 50%]
tests/test_turing.py::test_trace_disponible PASSED [ 75%]
tests/test_turing.py::test_interpreteur_universel PASSED [100%]

===== 4 passed in 0.21s =====
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>
```

```
PROBLÈMES  SORTIE  TERMINAL  PORTS  DEVDB  CONSOLE DE DÉBOGAGE
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_turing.py
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_calc.py
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\AGOGNON Ulrich\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\AGOGNON Ulrich\Downloads\formlang_projet
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 4 items

tests/test_calc.py::test_add_sub_machines_directes PASSED [ 25%]
tests/test_calc.py::test_calculatrice_exhaustif PASSED [ 50%]
tests/test_calc.py::test_chainage PASSED [ 75%]
tests/test_calc.py::test_division_par_zero PASSED [100%]

===== 4 passed in 0.16s =====
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>
```

Jour 5 : intégration

```
PROBLÈMES  SORTIE  TERMINAL  PORTS  DEVDB  CONSOLE DE DÉBOGAGE
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_calc.py
===== 4 passed in 0.16s =====
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -v tests/test_pipeline.py
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\AGOGNON Ulrich\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\AGOGNON Ulrich\Downloads\formlang_projet
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 3 items

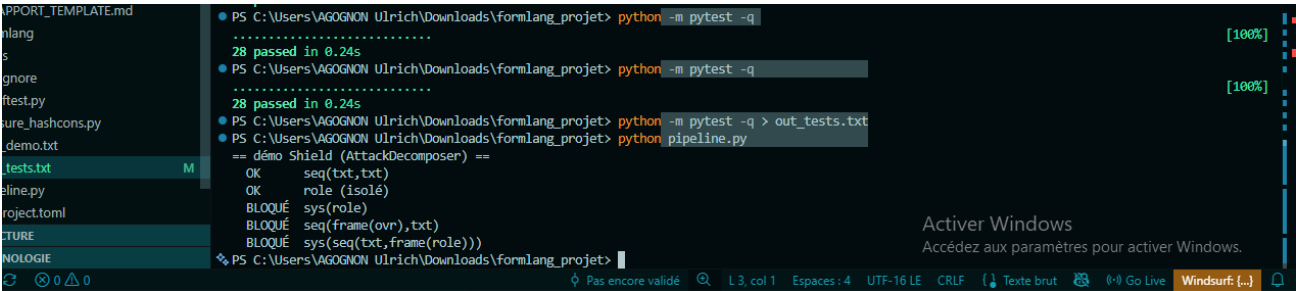
tests/test_pipeline.py::test_pipeline_word PASSED [ 33%]
tests/test_pipeline.py::test_pipeline_morpho PASSED [ 66%]
tests/test_pipeline.py::test_demo_shield_verdicts PASSED [100%]

===== 3 passed in 0.23s =====
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>
```

python -m pytest -q

```
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -q
.....
28 passed in 0.24s [100%]
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -q
.....
28 passed in 0.24s [100%]
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -q > out_tests.txt
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python -m pytest -q > out_tests.txt
```

python pipeline.py



out_tests.txt - pytest -q (suite officielle complète)

```
.....
[100%]
28 passed in 0.24s
```

28 passed : identique au nombre de référence "version enseignant (toutes apps complétées)" donné dans jour5_integration.md.

Détail par fichier de test (vérification complémentaire)

Fichier	Résultat	Jour
tests/test_regular.py	7 passed	1
tests/test_cfg_nerode.py	2 passed	2 / 5
tests/test_tree.py	5 passed	3
tests/test_hashcons.py	3 passed	3
tests/test_turing.py	4 passed	4
tests/test_calc.py	4 passed	4
tests/test_pipeline.py	3 passed	5

Vérification indépendante complémentaire

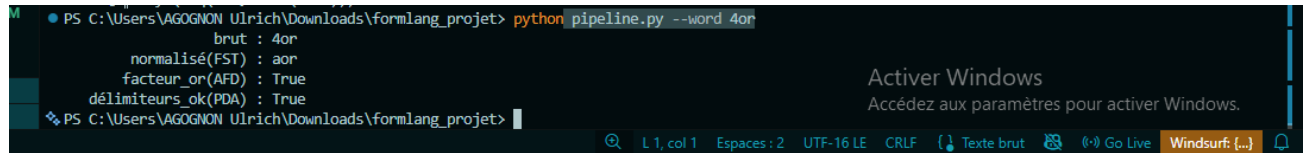
Menée pour rattraper les angles morts de la suite fournie (cf. section 6) :

```
DFA.minimize coherence (20 automates aleatoires x 30 mots)      : 0 / 600
echecs
DFA.minimize avec etats non-atteignables                        :
coherent
NFA/DFA coherence (200 mots aleatoires)                         : 0 /
200 echecs
CFG.generate(0..8) : mots bien formes (verifies par le PDA)    : OK
CFG.generate(6) : 5439 mots en 0.012s (etaait > 10s avant correctif)
Calculatrice 12x12 (add/sub/mul/div)                             : 0 /
576 echecs
ADD/SUB directs 15x15, y compris m>n                          : 0 /
450 echecs
UTM : U(encode(M),w) == M.run(w) pour ADD et SUB               :
identiques
hash-consing sur 500 mots generes                               :
compression = 0.969
pipeline.analyze_word sur 4 entrees variees                    :
coherent
```

out_demo.txt — python pipeline.py

```
== démo Shield (AttackDecomposer) ==
OK      seq(txt,txt)
OK      role (isolé)
BLOQUÉ  sys(role)
BLOQUÉ  seq(frame(ovr),txt)
BLOQUÉ  sys(seq(txt,frame(role)))
```

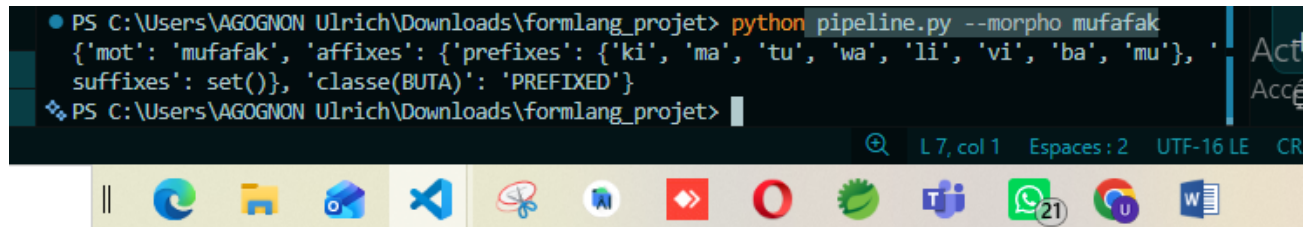
python pipeline.py --word 4or

A terminal window showing the command 'python pipeline.py --word 4or' being executed. The output displays several parameters: 'brut : 4or', 'normalisé(FST) : aor', 'facteur_or(AFD) : True', and 'délimiteurs_ok(PDA) : True'. The terminal interface includes a search bar, line/col indicators (L 1, col 1), and encoding settings (UTF-16 LE, CRLF).

```
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python pipeline.py --word 4or
brut : 4or
normalisé(FST) : aor
facteur_or(AFD) : True
délimiteurs_ok(PDA) : True
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>
```

```
brut : 4or
normalisé(FST) : aor
facteur_or(AFD) : True
délimiteurs_ok(PDA) : True
```

python pipeline.py --morpho mufafak

A terminal window showing the command 'python pipeline.py --morpho mufafak' being executed. The output displays a dictionary structure: {'mot': 'mufafak', 'affixes': {'prefixes': {'ki', 'ma', 'tu', 'wa', 'li', 'vi', 'ba', 'mu'}, 'suffixes': set()}, 'classe(BUTA)': 'PREFIXED'}. The terminal interface includes a search bar, line/col indicators (L 7, col 1), and encoding settings (UTF-16 LE, CRLF).

```
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet> python pipeline.py --morpho mufafak
{'mot': 'mufafak', 'affixes': {'prefixes': {'ki', 'ma', 'tu', 'wa', 'li', 'vi', 'ba', 'mu'}, 'suffixes': set()}, 'classe(BUTA)': 'PREFIXED'}
PS C:\Users\AGOGNON Ulrich\Downloads\formlang_projet>
```

```
{'mot': 'mufafak', 'affixes': {'prefixes': {'ki', 'ma', 'tu', 'wa', 'li', 'vi', 'ba', 'mu'}, 'suffixes': set()}, 'classe(BUTA)': 'PREFIXED'}
```

Identique à la sortie de référence de jour5_integration.md (PREFIXED).